

Deep Q-Network Control of the Unitree G1 Left Elbow

A Multi-Goal Reinforcement-Learning Study in MuJoCo and Gymnasium

Student	Viraj Dipakkumar Mistry
Student ID	9088985
Course	CSCN8020 - Reinforcement Learning
Selected model	Configuration A - epsilon decay 0.995
Repository	github.com/vir33mistry/CSCN8020_Assignment3

Executive summary

This project extends the completed Unitree MuJoCo G1 Primer with a student-written PyTorch DQN. The approved observation, actions, controller, reward, and success logic remain unchanged. Two controlled 600-episode CPU experiments differ only in epsilon decay. Both achieve 20/20 greedy successes across four benchmark goals. Configuration A is selected because it reaches a higher mean evaluation reward (13.1497) and lower mean final absolute error (0.0071 rad) than Configuration B.

Key result: the selected DQN achieves 100% success, exceeding the assignment's 80% threshold by 20 percentage points.

Policy	Success	Mean reward	Mean length	Mean final error
Config A	20/20	13.1497	20.75	0.0071 rad
Config B	20/20	13.0880	19.75	0.0136 rad
Rule-based	20/20	12.8666	24.00	0.0122 rad

Final validated environment: Python 3.12.13, Linux x86_64, PyTorch 2.7.1 CPU, Gymnasium 1.3.0, MuJoCo 3.10.0.

1. Environment and baseline verification

The primer provides a fixed-base G1 model, verified joint and actuator mappings, proportional-derivative control, MuJoCo bias-force compensation, and a compliant Gymnasium environment. The DQN replaces only the high-level hand-written policy.

Element	Approved definition
Observation	[elbow angle, angular velocity, goal, goal - angle]
Action 0	Decrease the internal controller target by 0.08 rad
Action 1	Hold the internal controller target
Action 2	Increase the internal controller target by 0.08 rad
Low-level control	PD torque plus qfric_bias compensation and torque clipping
Reward	- error ; +1 in tolerance; -0.05 for non-HOLD near goal; +10 on success
Success	error ≤ 0.04 rad for 8 consecutive environment steps
Time limit	150 environment steps

Termination and truncation

terminated=True means the task-defined success condition was reached. truncated=True means the 150-step limit ended the episode first. Both signals stop interaction. Replay stores them separately, and only true termination masks the Bellman bootstrap.

Pre-training baseline evidence

Goal	Episodes	Success	Mean reward	Mean length	Final error
-0.8	5	5/5	10.5215	27	0.0123
-0.4	5	5/5	15.2141	21	0.0132
+0.4	5	5/5	15.2111	21	0.0127
+0.8	5	5/5	10.5199	27	0.0106
Overall	20	20/20	12.8666	24.0	0.0122

- Gymnasium environment checker passed.
- Rule-based validation succeeded in every benchmark episode.
- The fixed-base model and approved controller were not redesigned.

2. Student-written DQN methodology

Q-network architecture

Layer	Dimensions	Activation	Purpose
Input	4	-	Approved observation vector
Hidden 1	4 -> 64	ReLU	Non-linear state features
Hidden 2	64 -> 64	ReLU	Higher-level interactions
Output	64 -> 3	None	Unconstrained action Q-values

The final layer deliberately has no softmax. DQN needs relative, unbounded estimates of discounted return rather than action probabilities. ReLU layers use Kaiming initialization; the final layer uses a small uniform initialization.

Replay and target network

- ReplayBuffer has bounded capacity 50,000 and seeded uniform sampling without replacement.
- Each transition stores state, action, reward, next state, terminated, and truncated.
- Learning begins only after 500 transitions and uses mini-batches of 64.
- The target network starts from the online weights and synchronizes every 250 updates.

Bellman target and optimization

$$y = r + 0.95 \times (1 - \text{terminated}) \times \max Q_{\text{target}}(\text{next_state}, \text{next_action})$$

The online network supplies $Q(\text{state}, \text{selected action})$ through gather. The target is computed under no_grad and is detached. Smooth L1 (Huber) loss, Adam with learning rate 0.001, and gradient-norm clipping at 10 are applied before the optimizer step.

Checkpoint support

Each checkpoint records network weights, target weights, optimizer state, hyperparameters, epsilon, optimization count, seed, device, and experiment metadata. Loading defaults to CPU and reconstructs the architecture before applying the saved state dictionary.

3. Training workflow and reproducibility

Parameter	Baseline value
Discount factor gamma	0.95
Learning rate	0.001
Mini-batch size	64
Replay capacity	50,000
Initial / minimum epsilon	1.00 / 0.05
Target-network update	Every 250 optimization steps
Warm-up	500 transitions
Episode limit	150 environment steps
Training goal range	[-0.8, +0.8] rad
Network	4 -> 64 ReLU -> 64 ReLU -> 3
Loss / optimizer	Huber / Adam

Controlled experiment

Configuration A uses epsilon decay 0.995; Configuration B uses 0.985. Both use 600 episodes, seed 8020, identical initial weights, common goal-seed policy, CPU execution, and the same hyperparameters. Only exploration decay changes.

Seed controls

- Python random, NumPy, and PyTorch are seeded with 8020.
- Replay sampling uses its own seeded NumPy generator.
- Each Gymnasium reset receives seed + episode index.
- Greedy evaluation uses the same fixed 20 seeds for every policy.

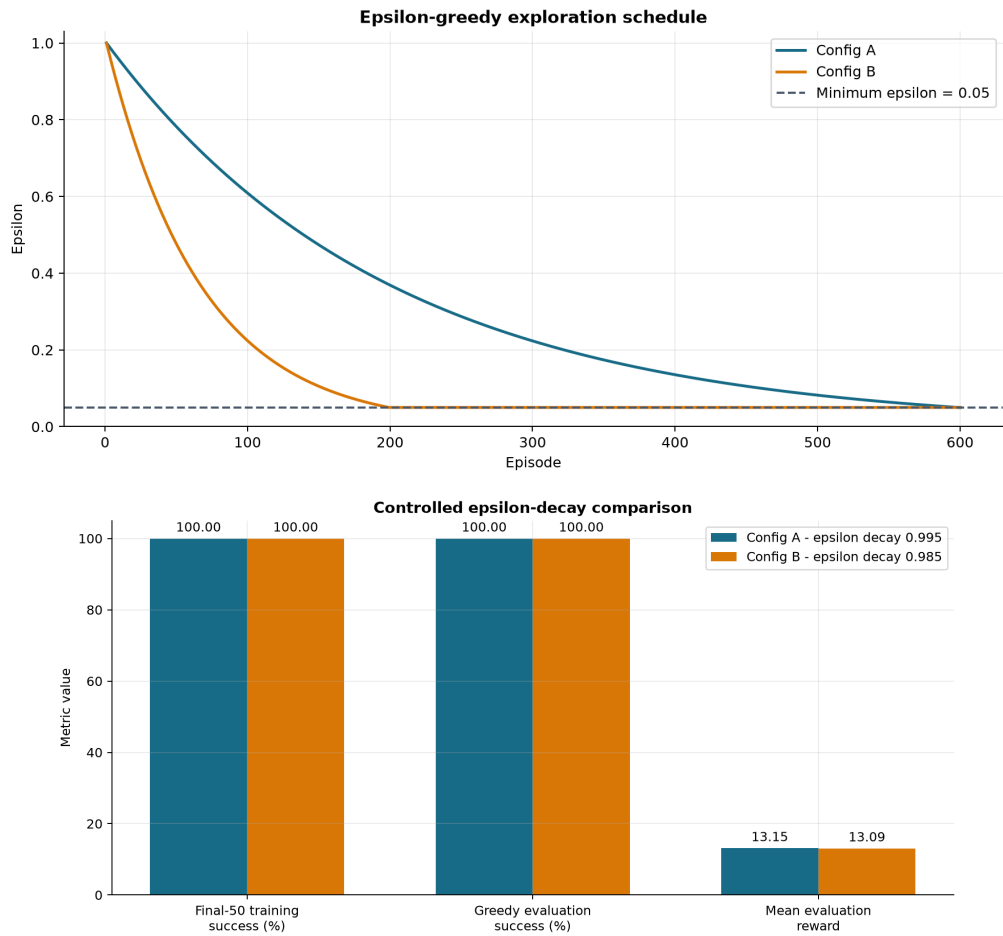
Measured execution

Configuration A completed in 53.70 seconds; Configuration B completed in 40.00 seconds. The combined required training time was about 1.56 minutes in the final validated CPU run. A per-configuration safety timer is included.

Training is headless. Rendering is isolated to saved-checkpoint demonstration scripts. CUDA can be used only when explicitly requested; CPU is the default and was the submitted validation path.

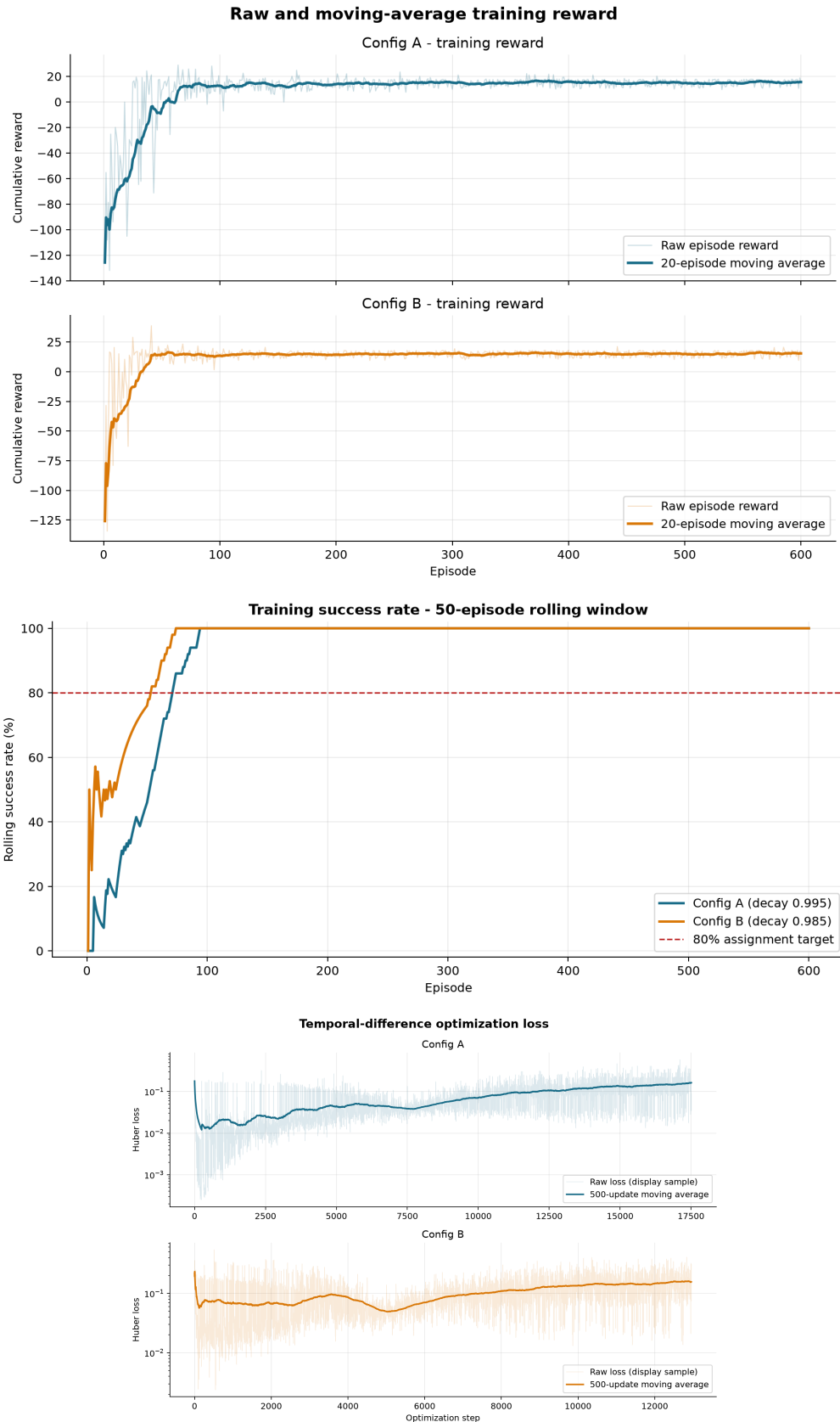
4. Exploration-decay comparison

Required metric	Config A: 0.995	Config B: 0.985
Episodes	600	600
Wall-clock training	53.70 s	40.00 s
Final epsilon	0.05	0.05
Final-20 mean reward	15.5566	15.3677
Final-50 train success	100%	100%
Greedy evaluation	20/20 (100%)	20/20 (100%)
Mean evaluation reward	13.1497	13.0880
Mean final error	0.0071 rad	0.0136 rad
Mean episode length	20.75	19.75
HOLD fraction	46.99%	11.39%



Config B reaches minimum exploration near episode 200. Config A explores longer, yet obtains the stronger reward and final-error evidence after both policies reach 100% success.

5. Training curves and stability

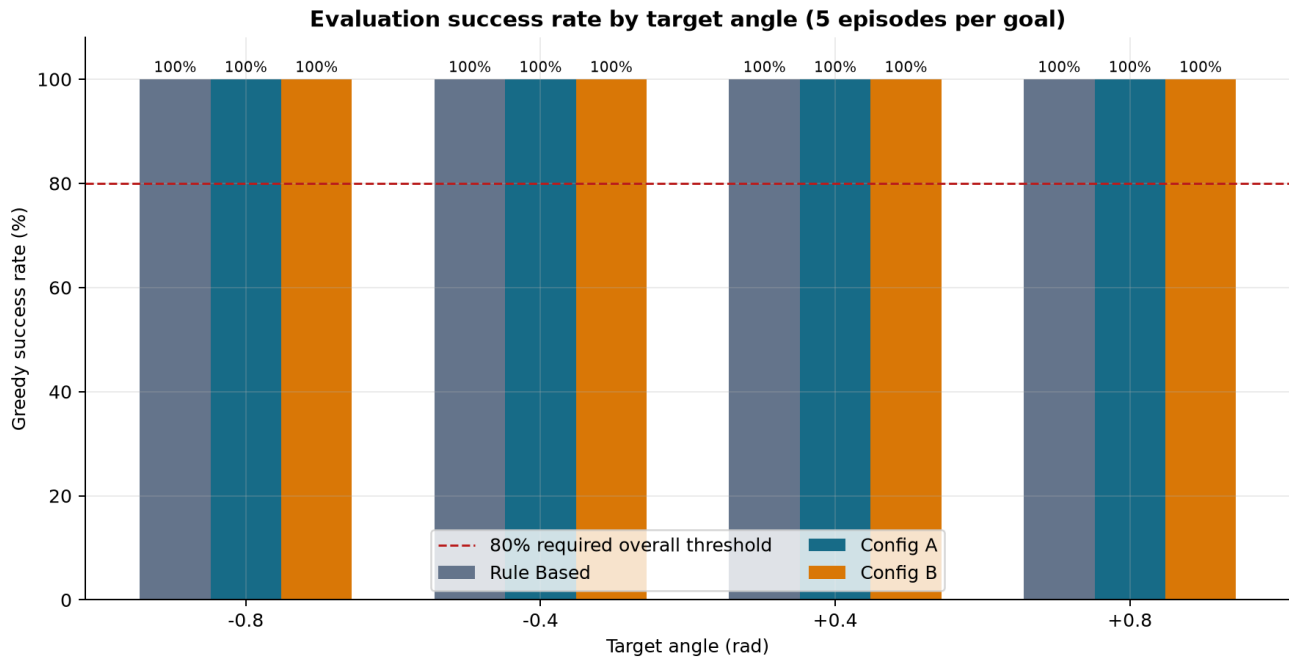


Random early episodes produce large negative returns. Both rolling success curves reach 100%. Huber loss is non-monotonic because Q-target scale and replay composition change as terminal bonuses become common; stable rewards and independent greedy evaluation confirm that learning remains effective.

6. Final greedy evaluation

Evaluation disables exploration (epsilon 0.0) and runs five episodes at each required goal. The selected checkpoint is loaded rather than retrained.

Goal	Episodes	Successes	Success rate	Mean reward
-0.8	5	5	100%	10.5898
-0.4	5	5	100%	15.5033
+0.4	5	5	100%	15.5175
+0.8	5	5	100%	10.9882
Overall	20	20	100%	13.1497



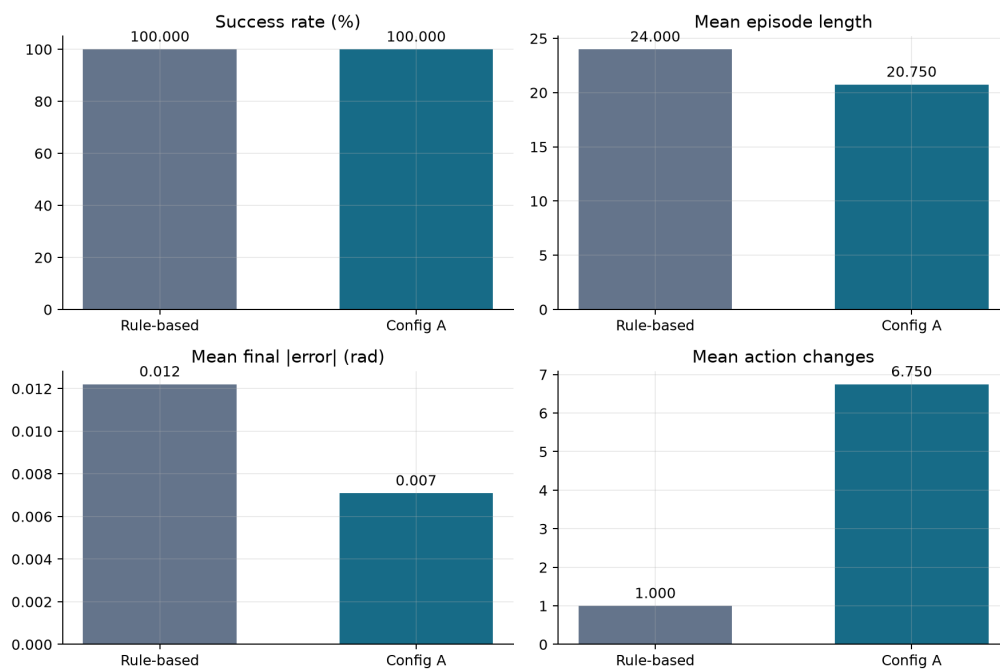
Threshold result: 20/20 successes = 100%. The required threshold is at least 16/20 = 80%. Configuration A exceeds the threshold and succeeds at every positive and negative target.

Per-goal mean final errors are 0.0093, 0.0050, 0.0102, and 0.0039 rad for -0.8, -0.4, +0.4, and +0.8 rad respectively. This supports multi-goal generalization rather than success at only one direction or magnitude.

7. Rule-based baseline versus selected DQN

Metric	Rule-based	Selected DQN
Successes / 20	20	20
Success rate	100%	100%
Mean cumulative reward	12.8666	13.1497
Mean episode length	24.00	20.75
Mean final absolute error	0.0122 rad	0.0071 rad
Mean action changes	1.00	6.75
HOLD action fraction	68.75%	46.99%
Main behaviour	Direct move, then stable HOLD	Learned corrective actions; faster finish

Rule-based baseline versus selected DQN



The rule-based policy is most sample efficient because it needs no learning, and it is more stable in action space. The DQN generalizes across all goals, finishes sooner, earns higher reward, and ends closer to the target, but changes action more often. It uses HOLD appropriately, though less consistently than the transparent baseline. A hand-written controller can excel in this simple task because it begins with perfect structural knowledge; DQN must infer that structure from finite experience.

8. Recommendation and conclusions

Recommendation: select Configuration A. Success rate is tied, so the decision uses multiple indicators: A has the higher final-20 training reward, higher evaluation reward, lower final error, and much greater HOLD use. Its additional measured training time is negligible relative to the five-hour limit.

Limitations

- Fixed-base, deterministic simulation controls one joint only.
- The primary comparison uses one training seed and 20 deterministic evaluation episodes.
- No disturbances, sensor noise, model mismatch, or physical-hardware effects are tested.
- The DQN changes actions more often than the rule-based policy.

Future improvements

- Repeat each configuration over several independent seeds.
- Evaluate a denser grid of unseen goals and controlled perturbations.
- With instructor approval, study action-switch regularization, Double DQN, or prioritized replay.
- Extend from one fixed-base joint to multi-joint and sim-to-real control.

Submission reproducibility

The repository includes explicit source, tests, metrics, plots, both experiment checkpoints, selected checkpoint, completed notebook, nine-page report, one-page submission PDF, exact commands, and a 2-minute 15-second saved-policy video. The interactive viewer script loads the checkpoint and uses epsilon 0.0 without retraining.

AI-use acknowledgement

Generative AI assistance supported scaffolding, debugging, test design, formatting, and draft writing. The student remains responsible for verification, understanding, and explanation.

References

1. Mnih et al. (2015), Human-level control through deep reinforcement learning, Nature 518, 529-533, doi:10.1038/nature14236.
2. PyTorch, Reinforcement Learning (DQN) Tutorial, docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html.
3. Farama Foundation, Handling Time Limits, gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/.
4. Google DeepMind, MuJoCo Python Documentation, mujoco.readthedocs.io/en/stable/python.html.
5. CSCN8020 Assignment 3 specification and G1 Primer Workshop.